OWASP Juice Shop VAPT Report

Local web application security assessment

Target	http://localhost:3000
Application	OWASP Juice Shop 20.3.0
Assessment date	27 October 2026 (IST)
Report version	2.0
Severity method	CVSS v3.1; score and vector per finding
Results	Six confirmed findings: 1 Critical and 5 Medium

Assessment focus

Authentication, sensitive data exposure, password hashing, object-level authorization, and cross-site scripting.

Intentionally vulnerable local training application using dummy accounts.

Executive summary

The assessment documented six confirmed findings in the local OWASP Juice Shop application: SQL injection authentication bypass, password hash disclosure in a login JWT, an unsalted MD5 password representation for the tested account, cross-account basket access, DOM-based XSS in search, and reflected XSS in order tracking. The evidence demonstrates an administrator session, exposed password-hash data, another account's basket contents, and browser script execution. The reflected XSS reproduction was rechecked and confirmed by the tester.

Findings overview

ID	Finding	CVSS 3.1	Severity
F-01	SQL Injection - Authentication Bypass	9.1	Critical
F-02	Password Hash Disclosure in JWT Payload	4.3	Medium
F-03	Weak Password Hashing - Unsalted MD5	5.3	Medium
F-04	IDOR - Unauthorized Shopping Basket Read	4.3	Medium
F-05	DOM-based XSS in Search	6.1	Medium
F-06	Reflected XSS in Order Tracking	6.1	Medium

Scope and environment

Target: http://localhost:3000. The assessment covered locally hosted OWASP Juice Shop 20.3.0 using dummy accounts. The version was confirmed by the tester from the npm startup output (juice-shop@20.3.0 start). The reviewed login and basket captures are dated 26-27 October 2026 UTC, corresponding to 27 October 2026 IST. Testing focused on login, authentication tokens, shopping baskets, search, and order tracking.

Severity basis

Severity uses CVSS v3.1, with calculated scores, complete vectors, and explicit metric assumptions for each finding. These are assessor-derived CVSS assessments using FIRST's standard. Scores describe technical severity in a reachable application deployment; the local lab remains at http://localhost:3000. The F-03 score explicitly models the documented F-02-to-F-03 hash-exposure and potential recovery scenario.

Validation

All six findings are confirmed for the behavior described. Validation is supported by HTTP captures, browser screenshots, DOM inspection, and the known-password MD5 comparison. Reflected XSS was additionally rechecked and confirmed by the tester. Suggested verification steps describe acceptance criteria for remediation.

Testing methodology

Phase 1 - Preparation

The authorized target was defined as the locally hosted Juice Shop lab at http://localhost:3000. Dummy accounts were used. Evidence requirements included saved request/response records or screenshots demonstrating the affected behavior.

Phase 2 - Reconnaissance

Login, basket, search, and tracking routes were identified through browser exploration, Burp Suite HTTP captures, and browser DOM inspection. Relevant input fields, route parameters, API calls, and authentication data were reviewed.

Phase 3 - Test Planning

Testing focused on SQL injection authentication bypass, password hash disclosure, weak password hashing, basket authorization, DOM-based XSS, and reflected XSS in order tracking. Relevant endpoints and parameters were selected from observed application behavior.

Phase 4 - Vulnerability Analysis

Login inputs and responses, JWT claims, basket identifiers, and XSS route parameters were examined. The paired basket captures provided a direct comparison between legitimate access and a modified request using the same authenticated session.

Phase 5 - Validation

HTTP records and screenshots were cross-checked against the stated findings. A known-password comparison confirmed that the disclosed test-account value matched its unsalted MD5 digest. XSS validation used visible script execution, DOM evidence, and the application challenge notification. Reflected XSS was rechecked and confirmed by the tester.

Phase 6 - Impact Analysis

Impact was assessed from the observed administrator session, disclosed password hash, cross-account basket data, and browser script execution. Potential consequences were evaluated in relation to confidentiality, integrity, and access control.

Phase 7 - Reporting

Six findings were documented with affected features, prerequisites, reproduction steps, evidence, impact, CVSS v3.1 scores and vectors, remediation recommendations, and suggested verification criteria.

CVSS scoring methodology

Standard and calculation

Scores use FIRST CVSS v3.1 Base equations, metric weights, and roundup-to-one-decimal rule. Each vector below reproduces its score. These are assessor-calculated assessments, with explicit scenarios rather than vendor-issued ratings.

ID / score	Vector
F-01 / 9.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
F-02 / 4.3	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
F-03 / 5.3	CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:N/A:N
F-04 / 4.3	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
F-05 / 6.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
F-06 / 6.1	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

Severity bands

None: 0.0 | Low: 0.1-3.9 | Medium: 4.0-6.9 | High: 7.0-8.9 | Critical: 9.0-10.0

Metric definitions

AV: attack vector; AC: attack complexity; PR: attacker privileges required; UI: other-user interaction; S: security scope; C/I/A: confidentiality, integrity, availability. Full metric choices and rationale accompany each finding.

Scoring context

The HTTP/application paths are scored as Network. Localhost is the lab deployment context and does not imply Internet exposure. F-01 assumes administrator access permits serious data access and modification. F-03 explicitly models F-02 exposure followed by potential password recovery; its score is conditional on that scenario. Scores are not added together.

Sources

FIRST CVSS v3.1 specification and user guide; Juice Shop 20.3.0 release. Links appear in the reference section. CVSS is owned by FIRST.Org, Inc. and used under its published terms.

F-01 | SQL Injection - Authentication Bypass

CVSS v3.1: 9.1 Critical | CWE-89 | Confirmed

`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Affected feature

POST /rest/user/login; JSON email parameter

Description

SQL syntax supplied in the login email field caused the application to issue an administrator authentication token. The response identified admin@juice-sh.op even though the request supplied the dummy customer email and customer password.

Prerequisites

Access to the local login endpoint.

Reproduction

1. Open the login form and capture POST /rest/user/login in Burp Suite.

2. Set the email value to the dummy email followed by the SQL suffix shown below, while retaining the dummy customer password.

3. Send the request and inspect the authentication object in the response.

4. Observe HTTP 200, umail admin@juice-sh.op, and bid 1. The browser screenshot separately shows the Login Admin challenge confirmation.

```
POST /rest/user/login
Content-Type: application/json

{
  "email": "ankit@gmail.com' OR 1=1--",
  "password": "Ankit@123"
}
```

Observed result

The saved response returns an administrator token. Its decoded data identifies user 1 with role admin. The browser also reports successful completion of the Login Admin challenge.

Impact

The injected login request produced an administrator session without the administrator password, crossing the application authentication boundary and granting the privileges associated with that account.

CVSS rationale

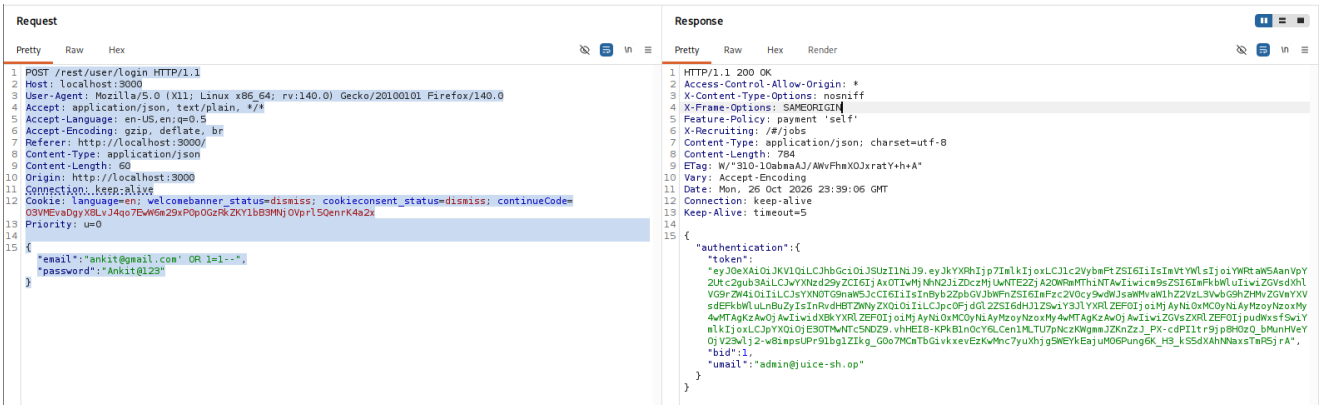
AV:N: the vulnerable login is reached through HTTP. AC:L: a single deterministic login payload is sufficient. PR:N/UI:N: neither prior access nor another user is needed. S:U: application and its database share the security authority. C:H/I:H: the issued administrator session is assessed as allowing serious access to and alteration of application data under that role. A:N: service disruption is outside the scored outcome.

High confidentiality and integrity impacts are inferred from administrator privileges; the evidence demonstrates administrator authentication. The score does not assume service shutdown or operating-system compromise.

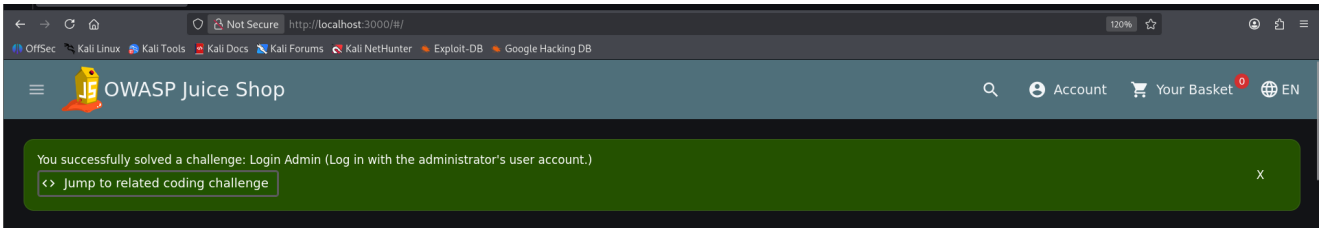
Remediation

Use parameterized queries or safe ORM query methods for all login data. Look up the account using bound values and verify its password with a password-hashing library. Apply least database privilege and keep database errors out of client responses.

F-01 | Supporting evidence



Injected login request and HTTP 200 response identifying the administrator.



Juice Shop reports successful completion of the Login Admin challenge.

Evidence files

evidence/01-sqli-auth-bypass/01-sqli-auth-bypass_request_response.md

evidence/01-sqli-auth-bypass/screenshots/01_sqli_auth_bypass_request_response.png

evidence/01-sqli-auth-bypass/screenshots/01_sqli_auth_bypass_successfull_admin_login.png

F-02 | Password Hash Disclosure in JWT Payload

CVSS v3.1: 4.3 Medium | CWE-201 | Confirmed

`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N`

Affected feature

POST /rest/user/login; authentication.token -> data.password

Description

A normal login response includes a signed JWT whose readable payload contains the test account password hash. The token serializes password-related data that is unnecessary for client authentication.

Prerequisites

A successful login to a dummy customer account. Decoding the payload does not require the signing key.

Reproduction

1. Log in with the dummy account using the normal POST /rest/user/login request.

2. Copy authentication.token from the HTTP 200 response.

3. Decode the middle Base64url segment of the JWT locally as JSON.

4. Observe data.password alongside user ID 25, role customer, and basket ID 6.

```
{
  "data": {
    "id": 25,
    "password": "5de52ea9d787966e8b598720255a37a7",
    "role": "customer"
  },
  "bid": 6
}
```

Observed result

The normal login response and decoded-token screenshot expose the same password hash: 5de52ea9d787966e8b598720255a37a7. The hash is present in data.password within the readable JWT payload.

Impact

A holder of the token can extract the test account hash and attempt password guesses offline. The observed MD5 representation increases the consequence of disclosure and is assessed separately in F-03.

CVSS rationale

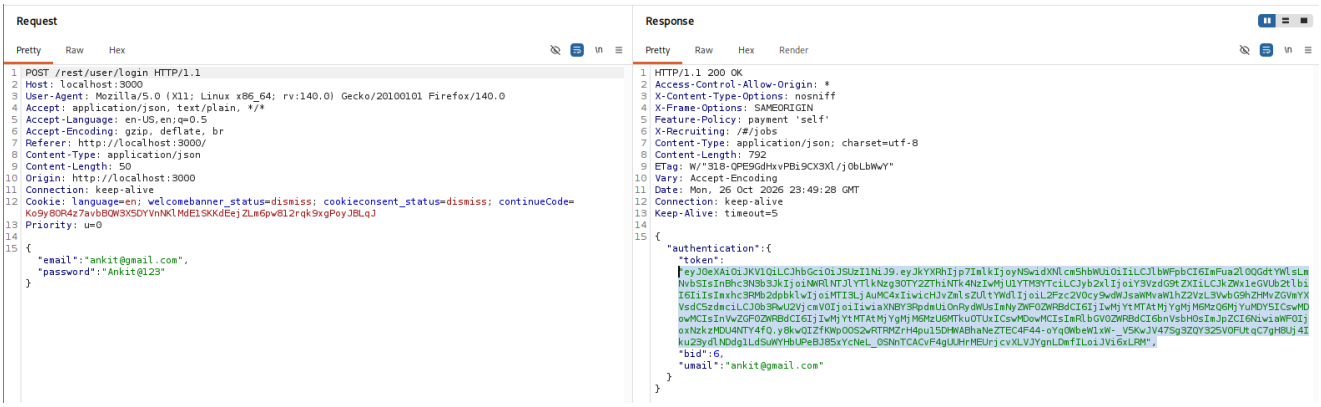
AV:N/AC:L: a normal HTTP login response discloses the value without a special race or configuration. PR:L: the assessed access path uses a valid customer account. UI:N: no separate user action is needed. S:U: exposure remains within the application. C:L: the response exposes the logged-in account password verifier. I:N/A:N: direct disclosure alone does not alter data or disrupt service.

This score covers the verified disclosure of the authenticated account hash. Password recovery and administrator bypass are separate findings, so their impacts are not added to this score.

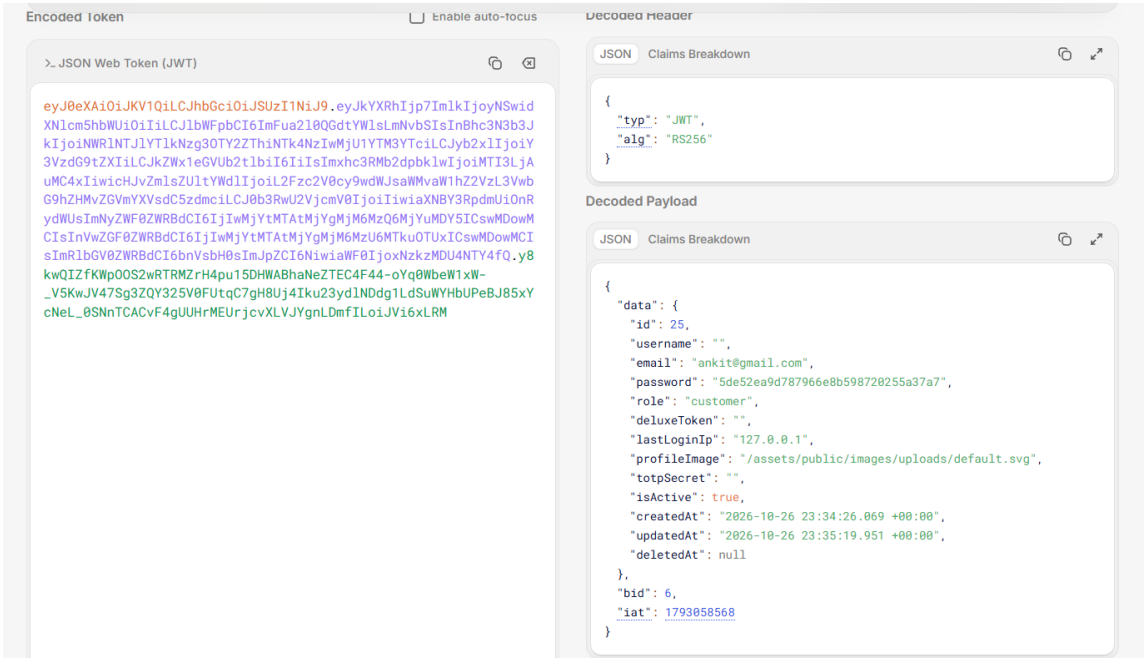
Remediation

Build JWT claims from an explicit allowlist of necessary identifiers and authorization claims. Exclude password hashes and other authentication secrets from token payloads and API serializers. JWT signing protects integrity and does not hide payload contents.

F-02 | Supporting evidence



Normal login response containing the JWT.



Decoded JWT contains data.password; the hash is readable without a signing key.

Evidence files

evidence/02 -Password-Hash-Disclosure/02 - Password-Hash-Disclosure_request_response.md

evidence/02 -Password-Hash-Disclosure/screenshots/02 - Password_Hash_Disclosure_request_response.png

evidence/02 -Password-Hash-Disclosure/screenshots/02 -Password_Hash_Disclosure_JWT_TOKEN_Decoded.png

F-03 | Weak Password Hashing - Unsalted MD5

CVSS v3.1: 5.3 Medium | CWE-916 | Confirmed for the tested account

`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:N/A:N`

Affected feature

Password representation in the tested customer account; disclosed JWT data.password

Description

The disclosed hash exactly matches MD5 of the known dummy password, calculated without a salt. This supports the weak-hashing finding for the tested account beyond a hash-format guess.

Prerequisites

The known dummy password from the normal login capture and the corresponding hash disclosed in F-02.

Reproduction

1. Extract data.password from the normal login token documented in F-02.

2. Review the hash-identifier screenshot, which lists MD5 among multiple possible formats.

3. Independently calculate MD5 over the UTF-8 bytes of the known dummy password from the same request.

4. Compare the calculated digest with the disclosed hash; the values match exactly. The added verification record documents this report-preparation check.

```
MD5(UTF-8 dummy password) = 5de52ea9d787966e8b598720255a37a7
Disclosed JWT password   = 5de52ea9d787966e8b598720255a37a7
Exact match             = True
```

Observed result

An exact known-password comparison confirms that the observed value is the unsalted MD5 digest of the tested password. This check was performed during report preparation; the original terminal screenshot alone only suggested possible algorithms.

Impact

An attacker who obtains this password representation can make rapid offline guesses and reuse precomputed values. The matched digest is a direct MD5 value without an adaptive work factor or per-password salt. F-02 provides an observed exposure route.

CVSS rationale

AV:N/PR:L: the assessment uses the observed authenticated network path to obtain verifier material. AC:H: successful password recovery depends on possessing a target verifier and a recoverable password, conditions beyond a guaranteed MD5 comparison. UI:N/S:U: no separate user action or authority change is required in the modeled scenario. C:H: recovery exposes a password secret. I:N/A:N: this score ends at password disclosure.

This is an explicit F-02-to-F-03 exposure-and-recovery scenario score, not a universal standalone score for the MD5 algorithm. The tested known-password match proves the representation; recovery of an unknown password is a potential consequence. The two findings describe separate controls and their scores are not additive.

Remediation

Replace direct MD5 password hashing with a maintained Argon2id implementation using unique random salts and an appropriate cost. Migrate legacy hashes by rehashing after successful login or requiring a password reset. Do not replace MD5 with another fast general-purpose hash.

F-03 | Supporting evidence



hash-identifier v1.2 proposes MD5 and other formats. The known-password comparison supplies the stronger algorithm check.

Known-password verification

```
MD5(UTF-8 dummy password) = 5de52ea9d787966e8b598720255a37a7
Disclosed JWT password   = 5de52ea9d787966e8b598720255a37a7
Exact match              = True
```

The known dummy password from the saved login capture produces the disclosed MD5 digest.

Evidence files

evidence/02 -Password-Hash-Disclosure/02 - Password-Hash-Disclosure_request_response.md

evidence/03 -Weak_Password_Hashing/03-md5-known-password-verification.md

evidence/03 -Weak_Password_Hashing/screenshots/03 -Weak_Password_Hashing_MD5.png

F-04 | IDOR - Unauthorized Shopping Basket Read

CVSS v3.1: 4.3 Medium | CWE-639 | Confirmed

`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N`

Affected feature

GET /rest/basket/{id}; basket ID path parameter

Description

Changing only the basket ID allowed customer user 26 to read a basket owned by customer user 25. The same bearer token was retained in both requests.

Prerequisites

An authenticated dummy customer session for user 26, with basket 7, and a second test account owning basket 6.

Reproduction

1. Authenticate as user 26 and capture GET /rest/basket/7.

2. Observe HTTP 200 with data.id 7 and UserId 26. This basket contains Banana Juice (1000ml).

3. Retain the same Authorization bearer token and cookie token, changing only the path to /rest/basket/6.

4. Send the modified request and observe HTTP 200 with data.id 6 and UserId 25. The returned basket contains Apple Juice (1000ml).

```
Authenticated principal: user 26 (token bid 7)

GET /rest/basket/7 -> 200, UserId 26, Banana Juice
GET /rest/basket/6 -> 200, UserId 25, Apple Juice

Authorization token: unchanged
```

Observed result

The legitimate and modified captures use the same token identifying user 26. The modified response returns a basket owned by user 25, demonstrating an object-level authorization failure.

Impact

An authenticated customer can read another customer basket, including its products, quantities, and ownership metadata. The captured requests demonstrate a cross-account confidentiality breach using an unchanged authentication token.

CVSS rationale

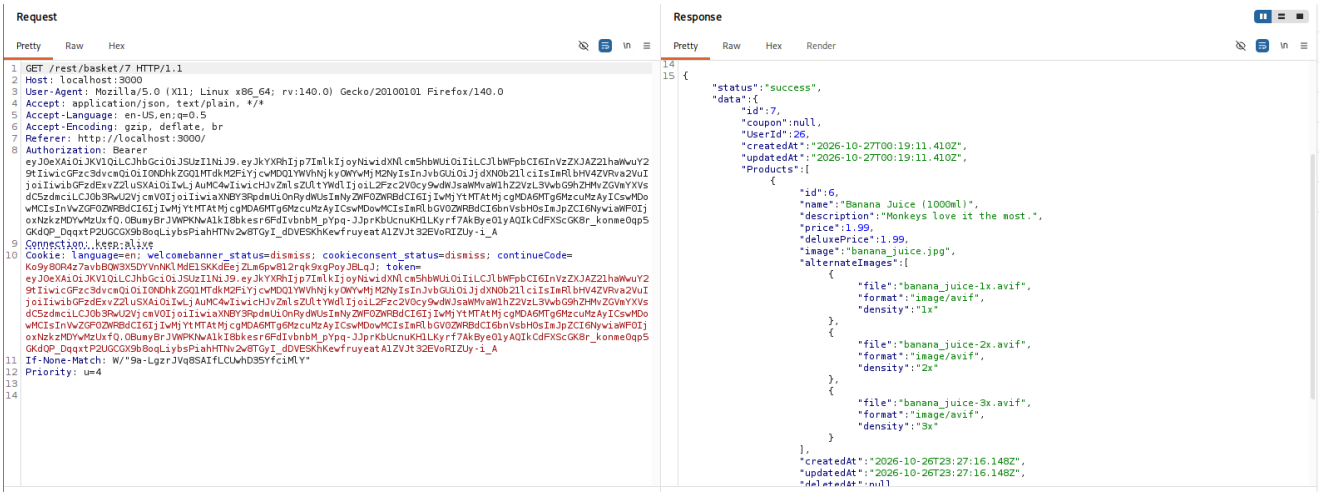
AV:N/AC:L: changing one HTTP path identifier returns another basket. PR:L: a customer token is used. UI:N: no other user participates. S:U: the crossed account boundary is within the same application authority. C:L: another account products, quantities, and basket metadata are disclosed. I:N/A:N: the assessed operation is a read.

The vector represents the demonstrated cross-account basket read. Integrity and service-availability impacts are not inferred from a successful GET request.

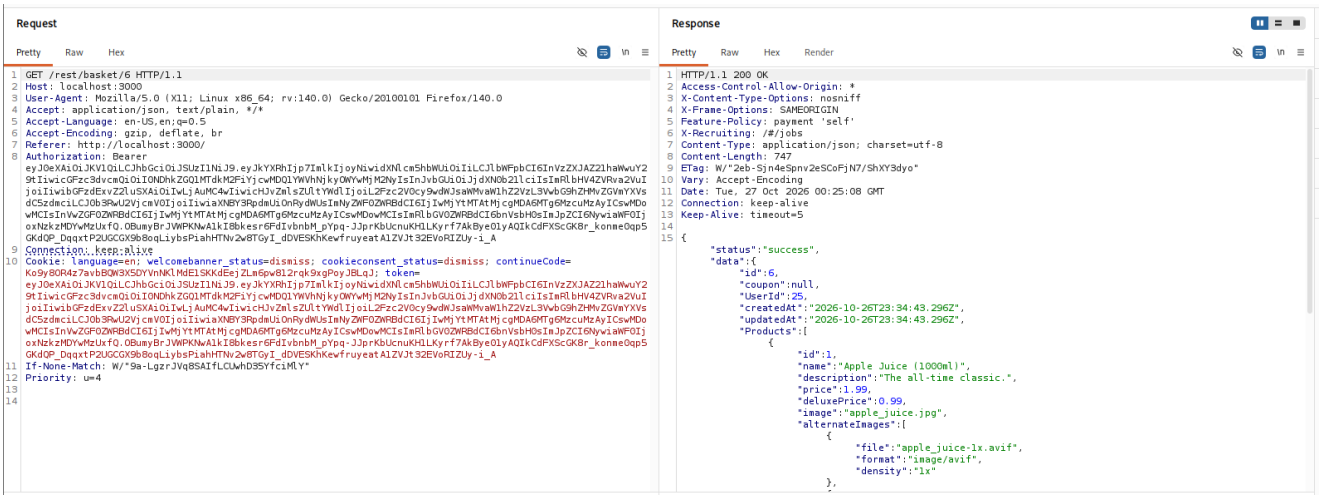
Remediation

Enforce ownership on the server for each basket request. Scope lookups to the authenticated principal and requested basket, deny access to other users unless an explicit authorized role permits it, and apply the same checks to related read and write endpoints.

F-04 | Supporting evidence



Baseline: user 26 reads basket 7, which belongs to user 26.



Modified request: the same user token reads basket 6, which belongs to user 25.

Evidence files

evidence/04-basket-idor/04-basket_idors_request_response_legitimate_request.md

evidence/04-basket-idor/04-basket_idors_request_response_modified_request.md

evidence/04-basket-idor/screenshots/04_basket_idor_Legitimate_Basket.png

evidence/04-basket-idor/screenshots/04-basket_idor_modified_request.png

F-05 | DOM-based XSS in Search

CVSS v3.1: 6.1 Medium | CWE-79 | Confirmed

`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Affected feature

/#/search?q=...; client-side q parameter and searchValue DOM element

Description

The search route renders attacker-controlled q input as an HTML iframe with a javascript URL. The supplied proof-of-concept executes an alert in the browser.

Prerequisites

A browser visiting the local Juice Shop search route with the supplied input. The evidence shows an existing application session.

Reproduction

1. Open the search route and supply the iframe payload shown below as q.

2. Submit the search or navigate to the corresponding hash-route URL.

3. Observe the browser alert displaying xss.

4. Inspect the DOM. The screenshot shows the iframe inside span#searchValue under app-search-result.

```
<iframe src="javascript:alert('xss')">
```

```
/#/search?q=%3Ciframe%20src%3D%22javascript%3Aalert%28%27xss%27%29%22%3E
```

Observed result

The execution screenshot shows the alert, and the inspector screenshot shows an executable iframe inserted into the search result heading. The q input is part of the URL fragment, which is handled by the client application.

Impact

A crafted search link can cause browser script execution when opened in the application. The captured proof of concept displays an alert. The same execution capability could allow page manipulation or actions under the affected browser session.

CVSS rationale

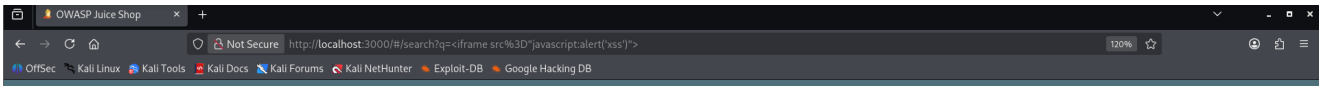
AV:N/AC:L: the crafted application URL provides a repeatable browser payload. PR:N: the attacker supplies a link without needing application privileges; the victim session is a separate matter. UI:R: the victim must open or submit the crafted input. S:C: the application flaw affects the victim browser security scope. C:L/I:L: injected script can access or change content available to that page. A:N: service availability is unaffected by the assessed outcome.

The alert and injected DOM node demonstrate script execution. Limited client-side confidentiality and integrity impacts are reasonable consequences of that execution; broad account takeover is not assumed.

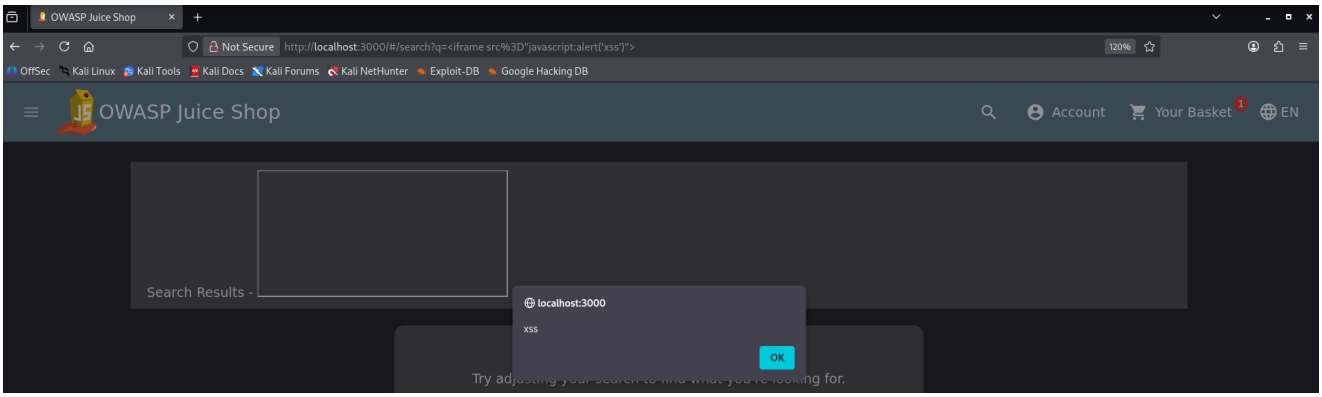
Remediation

Render search terms as text through framework interpolation or textContent. Avoid inserting user input into raw HTML and avoid sanitizer-bypass APIs. Where rich HTML is required, use a maintained sanitizer and enforce safe element, attribute, and URL-scheme policies.

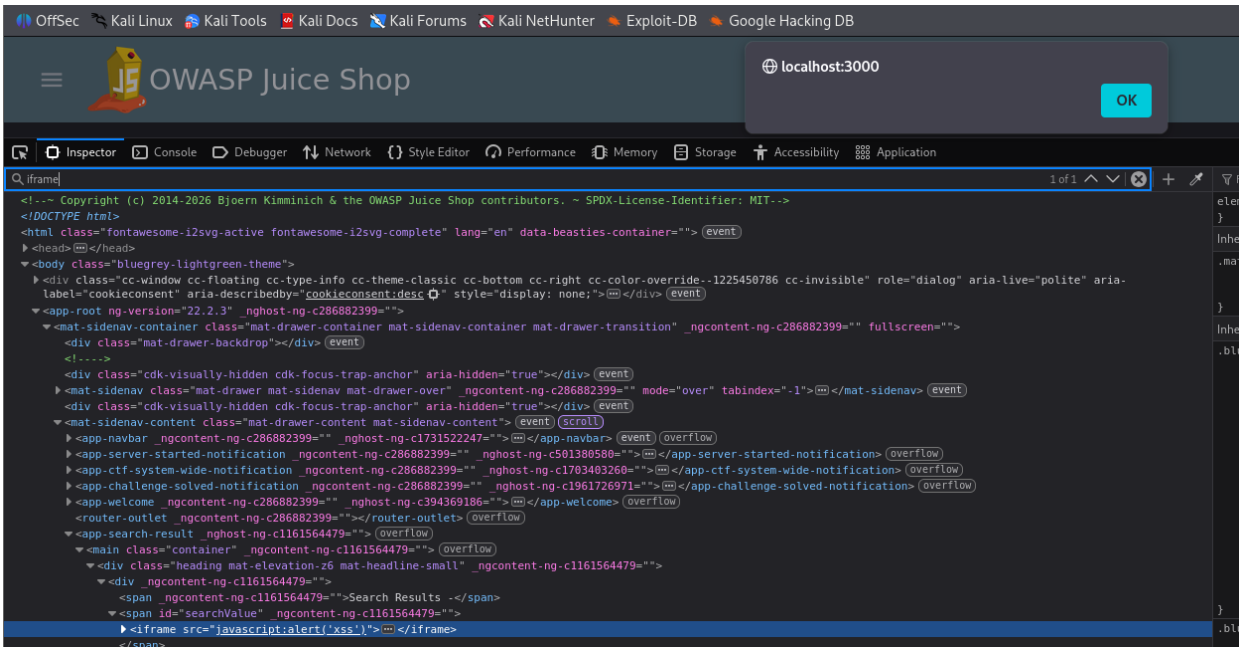
F-05 | Supporting evidence



Search URL contains the iframe proof-of-concept.



The browser executes the search payload and displays xss.



The inspector shows the iframe inside span#searchValue.

Evidence files

evidence/05_dom_xss/screenshots/05_dom_xss_URL_Payload.png

evidence/05_dom_xss/screenshots/05_dom_xss_JavaScript_Execution.png

evidence/05_dom_xss/screenshots/05_dom_xss_Inspect_Element.png

F-06 | Reflected XSS in Order Tracking

CVSS v3.1: 6.1 Medium | CWE-79 | Confirmed

`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Affected feature

/#/track-result?id=...; id route parameter

Description

Reflected XSS was confirmed through the order-tracking route id parameter. An iframe payload executed JavaScript and displayed an xss alert. The screenshots show the payload, browser execution, and Juice Shop Reflected XSS challenge confirmation. The tester confirmed successful reproduction during a subsequent check.

Prerequisites

A browser accessing the local Juice Shop application and its order-tracking route.

Reproduction

1. Open the local application and set the track-result route id value to the iframe payload shown below.
2. Submit the URL by pressing Enter.
3. Observe the xss browser alert.
4. Review the inspector screenshot showing the Juice Shop notification for the Reflected XSS challenge.

```
<iframe src="javascript:alert(`xss`)">
```

```
/#/track-result?id=%3Ciframe%20src%3D%22javascript%3Aalert%28%60xss%60%29%22%3E
```

Observed result

The tracking payload executed and displayed the xss browser alert. Juice Shop also reported successful completion of the Reflected XSS challenge. Successful reproduction was rechecked and confirmed by the tester.

Impact

A crafted tracking URL can execute attacker-controlled JavaScript in the application browser context when opened by a user. The demonstrated proof of concept displays an alert; the execution capability could also affect page content or actions permitted to the browser session.

CVSS rationale

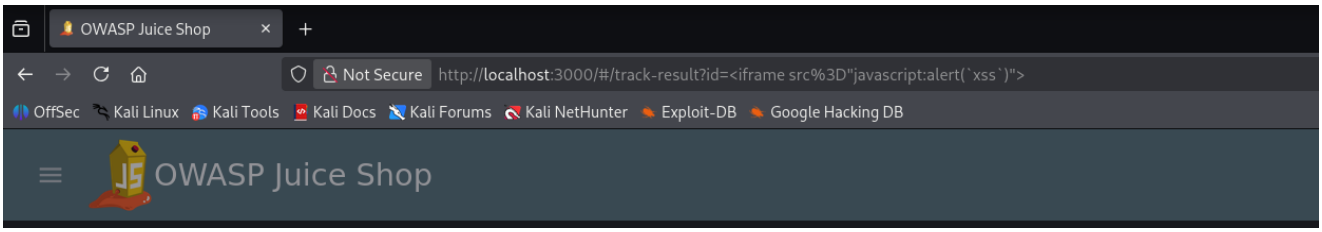
AV:N/AC:L: a crafted tracking link triggers the confirmed payload. PR:N: the attacker can construct the link without an application account. UI:R: another user must open it. S:C: browser data and behavior are impacted across the application-to-client authority boundary. C:L/I:L: execution permits limited access to or modification of page/session-accessible content. A:N: the scored effect does not interrupt the service.

The finding remains confirmed based on supplied screenshots and the tester recheck. The severity models client-side script execution requiring victim interaction.

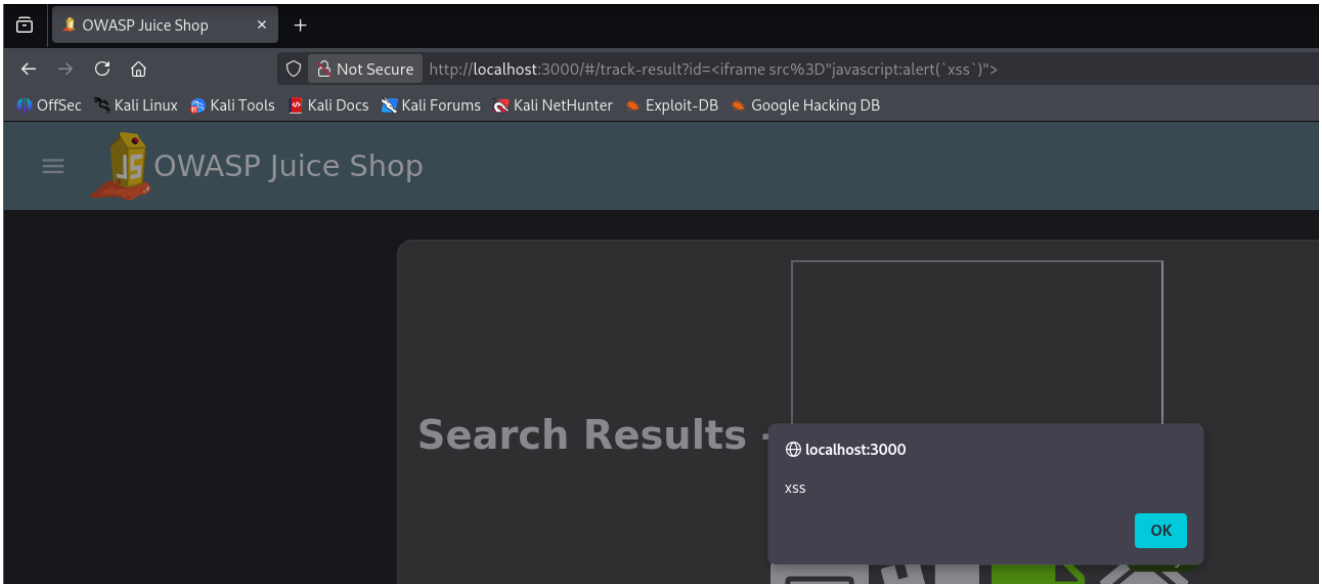
Remediation

Validate tracking identifiers against the expected identifier format on the server. Return untrusted values as data and render them with safe text bindings. Apply context-appropriate encoding at rendering boundaries. Use a restrictive Content Security Policy as defense in depth.

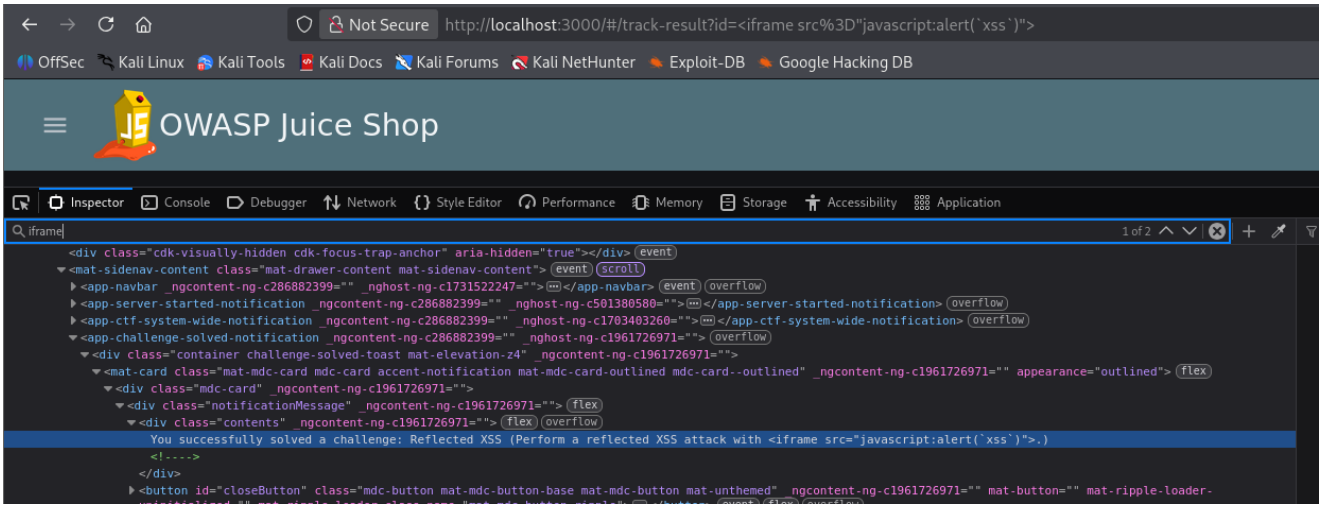
F-06 | Supporting evidence



The order-tracking URL contains the id payload.



The tracking payload triggers the xss browser alert.



The inspector shows the Juice Shop Reflected XSS challenge confirmation.

Evidence files

evidence/06_reflected_xss/screenshots/06_relfected_xss_url_payload.png

evidence/06_reflected_xss/screenshots/06_relfected_xss_javascript_execution.png

evidence/06_reflected_xss/screenshots/06_reflected_xss_inspect_element.png

Remediation recommendations

1 - Immediate | F-01

Parameterize login queries and verify passwords through a maintained library.

Completion criterion: The injected request issues no session; valid login remains functional.

2 - High priority | F-04

Enforce server-side basket ownership for reads and related operations.

Completion criterion: The same user cannot read the other account basket.

2 - High priority | F-02 / F-03

Remove password hashes from JWT claims and migrate MD5 representations to salted Argon2id.

Completion criterion: Tokens contain no password data; password hashes use an adaptive salted format.

2 - High priority | F-05 / F-06

Render untrusted input as text; validate tracking IDs and review HTML/URL sinks.

Completion criterion: Both route payloads are inert and valid search and tracking inputs remain functional.

3 - Defense in depth | F-01 / F-05 / F-06

Add least privilege, safe error handling, and a restrictive CSP after fixing primary causes.

Completion criterion: The primary security controls remain effective without relying solely on extra layers.

Suggested verification

F-01: Replay the captured payload and verify that no authentication token is issued. Confirm that valid dummy credentials still work and that ordinary invalid credentials fail. Review the query implementation for parameter binding.

F-02: Perform a normal login and decode the returned token. Verify that no password or password hash is present, while required authentication and authorization features still function.

F-03: Verify that the stored representation uses a modern password-hashing format. Two test accounts with the same password should have different salted hashes, and the raw MD5 digest should no longer match the stored value. Confirm valid login still succeeds.

F-04: User 26 should retain access to basket 7 and receive a consistent 403 or non-disclosing 404 for basket 6. Repeat in the opposite direction using user 25, and check related basket operations under the same authorization policy.

F-05: Replay the captured search payload and confirm that no iframe is created and no script runs. The input should appear as inert text or be safely rejected, while ordinary searches continue to work.

F-06: After remediation, open the captured tracking payload URL in a fresh tab or after a full reload. Confirm that the input is rejected or rendered as inert text, no script executes, and valid tracking identifiers continue to work.

Tools, references, and evidence notes

Tools used

Burp Suite was used for login and basket HTTP request/response analysis. Firefox 140.0 is recorded in the request User-Agent. Browser developer tools supported DOM inspection, a JWT decoder displayed the token claims, and hash-identifier v1.2 supplied hash-format candidates. Python hashlib/base64 and .NET MD5 calculations verified the known dummy password against the disclosed value during report preparation.

References

OWASP SQL Injection Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

RFC 7519 - JSON Web Token
https://www.rfc-editor.org/rfc/rfc7519

OWASP Password Storage Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Insecure Direct Object Reference Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

OWASP DOM based XSS Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html

OWASP Cross Site Scripting Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Pwning OWASP Juice Shop - Cross Site Scripting
https://pwning.owasp-juice.shop/companion-guide/latest/part2/xss.html

FIRST CVSS v3.1 specification
https://www.first.org/cvss/v3.1/specification-document

FIRST CVSS v3.1 user guide
https://www.first.org/cvss/v3.1/user-guide

Juice Shop 20.3.0 release
https://github.com/juice-shop/juice-shop/releases/tag/v20.3.0

Evidence handling

Source evidence files retain their original content. Report excerpts identify the relevant test values. The PDF reproduces the screenshots in their original colors. Captures use dummy lab account details and tokens.